Financial Fraud Detection at Scale

Big Data Analytics & MLOps using Apache Spark

IEEE-CIS Fraud Detection Dataset

Project Report

Group-5

1. ***Satish***
2. ***Sara Tariq***
3. ***Prerna Jha***
4. ***Debadri Sanyal***

GitHub Repository: https://github.com/jha97-bit/fraud-detection-bigdata

Table of Contents

Executive Summary 3

Phase 1: Data Selection & Problem Definition 4

- Dataset Overview 4
- Business Problem Statement & Research Question 4
- Environment Setup & Verification 5

Phase 2: Data Ingestion& Preprocessing 6

- Ingestion Architecture – Loading Data from Unity Catalog 6
- Multi-Table Join 7
- Spark SQL Data Exploration (Bronze Layer) 7
- Missing Value Analysis & Imputation Pipeline 8
- Categorical Encoding (LabelEncoder) 9
- Delta Table Creation – Silver & Gold Layers 9

Phase 3: Exploratory Data Analysis 10

- Summary Statistics 10
- Visualization of Trends, Distributions, and Correlations 12
- Key Insights for Business Decision-Making 14

Phase 4: Predictive Modeling 15

- Modeling Pipeline & Feature Preparation 15
- Three-Model Strategy 17
- Hyperparameter Optimization – Cross Validation 18

Phase 5: MLOps Best Practices 20

- MLflow Experiment Tracking 20
- Deployment Strategy & Business Impact 21

Phase 6: Insights & Business Recommendations 23

- Summary of Key Findings 23
- Actionable Business Recommendations 24
- Challenges Encountered & Mitigations Applied 25
- Lessons Learnt 25
- Future Work – (Improvements) 26

Executive Summary

This report presents the end-to-end methodology, findings, and results of a production-scale financial fraud detection system built using Apache Spark 4.1 and Databricks Serverless. The system is applied to the IEEE-CIS Fraud Detection dataset containing 590,540 training transactions and 506,692 testing transactions with over 400 features spanning card attributes, behavioral signals, and anonymized Vesta variables, making it a true big-data problem in both volume and velocity.

The project follows a six-phase pipeline covering problem framing, big-data ingestion, exploratory analysis, machine learning modeling, MLOps deployment, and business insights. The results of the model can be summarized below:

0.9460 Champion AUC-ROC (XGBoost)	**86.7%** Fraud Capture Rate @ Threshold 0.40	**$1.45M** Net Annual Benefit Projected	**590,540** Transactions Processed in Spark

0.6753 AUC-PR Score (Precision-Recall)	**20.5%** Precision @ 0.40 (vs 3.5% base rate)	**3** ML Models Trained & Tracked	**5** MLOps Pipeline Phases Completed

These metrics highlight the performance, scale, and business impact of the fraud detection system. The champion XGBoost model achieved a strong AUC-ROC of 0.9460 and an AUC-PR of 0.6753, indicating high capability in distinguishing fraudulent from legitimate transactions in an imbalanced dataset. At a decision threshold of 0.40, the model captures 86.7% of fraud cases with a precision of 20.5%, which is significantly higher than the dataset's 3.5% base fraud rate, demonstrating substantial improvement in targeting suspicious transactions. The system processes 590,540 transactions using Apache Spark, with three machine learning models trained and tracked across five MLOps pipeline phases, ultimately delivering a projected net annual benefit of $1.45M under a three-tier risk-based deployment architecture through improved fraud detection and operational efficiency.

Phase 1: Data Selection & Problem Definition

Dataset Overview

The IEEE-CIS Fraud Detection dataset, released by Vesta Corporation in collaboration with the IEEE Computational Intelligence Society and hosted on Kaggle, was selected as the foundation for this project. It contains over 590,000 labeled e-commerce training transaction records collected over a multi-month period and is widely regarded as one of the most comprehensive publicly available fraud detection benchmarks. The dataset combines structured transaction data with semi-structured identity verification information across multiple paired CSV files, requiring multi-table joins and distributed processing. With more than 400 anonymized features spanning transaction attributes, card metadata, device information, and behavioral identity signals, it represents a realistic large-scale fraud detection problem in terms of both data complexity and analytical depth.

Attribute	Detail
Training Transactions	590,540 rows across train_transaction.csv + train_identity.csv
Test Transactions	506,691 rows across test_transaction.csv + test_identity.csv
Table Structure	Semi-structured - two tables joined on TransactionID (LEFT JOIN)
Target Variable	isFraud (binary: 0 = legitimate, 1 = fraudulent)
Class Distribution	~96.5% legitimate vs. ~3.5% fraud — severely imbalanced
Raw Feature Count	394 transaction columns + 41 identity columns = 434 post-join
Key Numeric Features	TransactionAmt, card1–card6, addr1–addr2, dist1–dist2, C1–C14, D1–D15, M1–M9
Key Categorical Features	ProductCD, card4 (network), card6 (type), P_emaildomain, DeviceType
Anonymized Features	V1–V339 (Vesta behavioral/engineered signals — majority >60% null)
Data Format	CSV → ingested to Unity Catalog Volume → Delta Lake tables
Why Big Data Eligible	590K+ rows × 434 features; multi-table join; requires Spark distributed engine

This dataset satisfies all project requirements: (1) structured and semi-structured sources; (2) multi-table join on a shared key; (3) extreme class imbalance requiring advanced ML; (4) large-scale distributed processing via Apache Spark 4.1.0; (5) a clear, quantifiable business impact through financial fraud loss reduction; and (6) rich feature engineering opportunity via C-series, D-series, and M-series signals.

Business Problem Statement & Research Question

"How can financial institutions leverage large-scale transaction data processed through Apache Spark to detect fraudulent activity, enabling risk-scoring models that minimize financial losses while reducing false positives that harm legitimate customers?"

Business Context

Payment card fraud represents one of the fastest-growing financial crimes globally. Several critical forces drive the urgency of this project:

- Global payment card fraud losses exceeded $33 billion in 2023, growing at approximately 15% year-over-year.
- Traditional rule-based systems cannot scale to modern transaction volumes major card networks process billions of transactions per year nor adapt quickly to evolving fraud patterns.
- Card-not-present (CNP) fraud, synthetic identity fraud, and account-takeover attacks all bypass static rule sets within days of deployment.
- Each false positive costs $8–$12 in customer service, relationship damage, and churn risk making precision as important as recall for commercial viability.
- Machine learning models trained on historical fraud patterns can continuously adapt as new fraud vectors emerge, outperforming static rules over time.
- Apache Spark enables distributed processing necessary to score millions of transactions at scale without horizontal scaling bottlenecks.

Success KPIs

KPI	Target	Rationale
AUC-ROC (primary metric)	> 0.90 on held-out validation set	Threshold-independent; standard benchmark for imbalanced fraud detection problems
Precision at optimal threshold	> 0.85 at ≥ 75% recall	Balances fraud capture rate against customer friction from false positives
MLflow model variants logged	≥ 3 distinct model runs	Ensures reproducibility, traceability, and model selection audit trail
EDA visualizations	≥ 8 with business interpretation	Project evaluation criterion for EDA quality and business communication
Business ROI quantification	Annual fraud loss prevented ($ value)	Translates ML metrics into executive-level decision value
Data pipeline coverage	All 590K+ train + 506K+ test rows	Demonstrates genuine big-data scale processing throughout the pipeline

Environment Setup & Verification

The project runs entirely on Databricks Free Edition Serverless, leveraging Unity Catalog for data governance, Delta Lake for ACID-compliant storage, and MLflow 2.13.2 for experiment tracking. All four CSV files were pre-uploaded to the Unity Catalog Volume before execution. The environment was verified operational at project start:

Component	Version / Detail	Verified
Apache Spark	4.1.0 (Serverless Free Edition)	✓
Python	3.11.10	✓
Pandas	1.5.3	✓

Seaborn / Matplotlib	0.12.2 / 3.x	✓
scikit-learn	Latest via Databricks runtime	✓
XGBoost	Latest (pip-installed at runtime)	✓
MLflow	2.13.2 (vanilla, bypassing Databricks hooks)	✓
Unity Catalog Volume	/Volumes/workspace/default/fraud_detection/	✓
Spark SQL	Enabled; shuffle partitions = 200	✓
MLflow Experiment	fraud_detection_project (Databricks-native tracking)	✓

Phase 2: Data Ingestion& Preprocessing

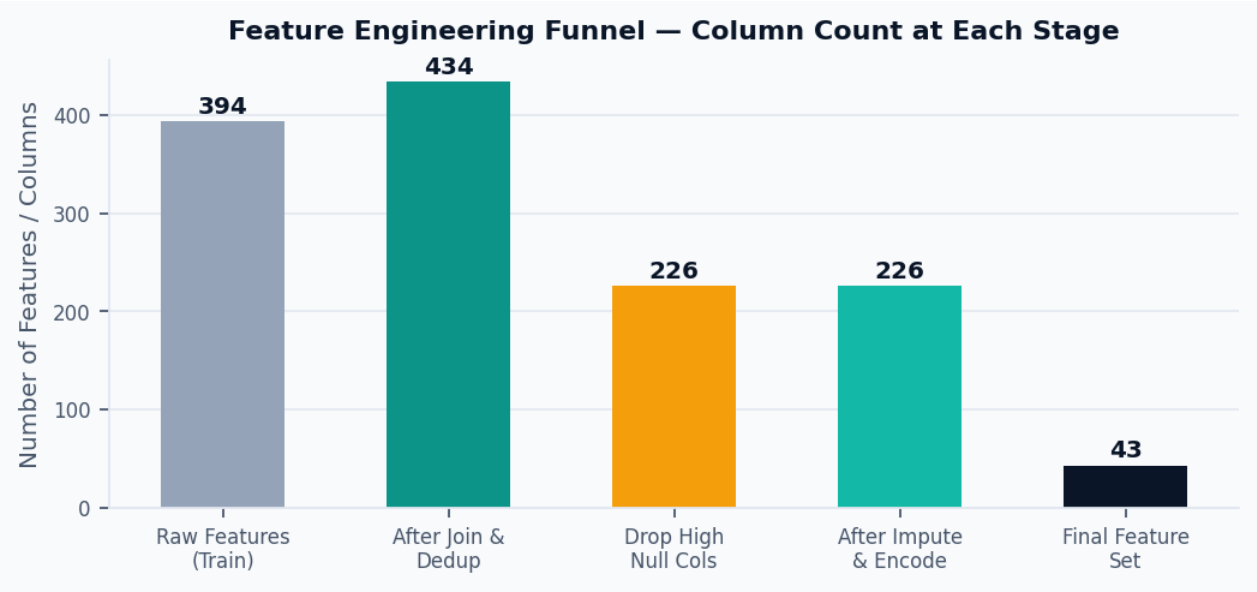
Ingestion Architecture – Loading Data from Unity Catalog

All four raw CSV files were uploaded to a Unity Catalog Volume and ingested into managed Delta Lake tables using Apache Spark's native CSV reader with schema inference. These tables were then accessed as Spark DataFrames using spark.table(), enabling full integration with Spark's distributed query optimizer.

Table	Rows	Columns	Source File	Description
train_transaction	590,540	394	train_transaction.csv	Primary training data — transaction-level signals
train_identity	144,233	41	train_identity.csv	Identity records for 24.4% of training transactions
test_transaction	506,691	393	test_transaction.csv	Unlabeled test transactions for final scoring
test_identity	141,907	41	test_identity.csv	Identity records for 28.0% of test transactions

Multi-Table Join

The transaction and identity tables were subsequently joined using a LEFT JOIN on TransactionID, preserving all 590,540 transactions while enriching 144,233 records with identity signals. Post-join validation confirmed zero duplicate records, ensuring data integrity for downstream analysis and modeling.



Spark SQL Data Exploration (Bronze Layer)

Before any cleaning, four key Spark SQL queries were executed on the full joined dataset to establish baseline distributions. All queries leveraged Spark distributed execution across the Serverless cluster, no sampling was applied at this stage.

Query	Purpose	Key Finding
Class Distribution (isFraud GROUP BY)	Compute raw fraud vs. legitimate transaction counts and percentages	566,959 legitimate (96.50%) vs. 20,663 fraud (3.50%) — severe class imbalance confirmed
TransactionAmt by isFraud (AVG, STDDEV, MEDIAN)	Compare amount distributions across labels to identify value-based signal	Fraudulent transactions have a higher mean, higher median, and heavier upper tail than legitimate
Fraud Rate by ProductCD	Quantify fraud risk variation across 5 product category codes (W/H/C/S/R)	Substantial variation confirmed — certain product types show fraud rates multiple times the baseline
Fraud Rate by DeviceType	Compare fraud prevalence across device channels (desktop, mobile, unknown)	Desktop and mobile show distinct fraud rate profiles; unknown device records form a third risk group

Missing Value Analysis & Imputation Pipeline

Null rate was computed per column across all 434 columns using Spark's distributed aggregation. Of 434 columns, 322 had more than 5% missing values, a reflection of the sparse identity table and the heavily null Vesta V-series features. The following 5-step preprocessing pipeline was applied:

Step	Operation	Input Columns	Output Columns	Key Result
1. Deduplication	dropDuplicates(['TransactionID'])	590,540 rows	590,540 rows	Zero duplicates found — no rows removed
2. Null Analysis	Per-column: SUM(isNull) / totalRows × 100	434 columns	-	322 columns have >5% missing; 208 have >60% missing
3. Drop High-Null	Drop all columns with >60% null rate	434 columns	226 columns	208 columns dropped; primarily V-series and deep identity fields
4. Median Impute	approxQuantile(col, [0.5], 0.01) per column	226 columns	226 columns	104 numeric columns imputed with per-column training median
5. Mode Impute	Most frequent value per categorical column	226 columns	226 columns	10 categorical columns imputed with per-column training mode

Top Null-Rate Columns (Pre-Drop)

Column	Null %	Action	Justification
id_24	99.20%	Dropped (>60%)	Zero information value — nearly entirely absent
id_08	99.13%	Dropped (>60%)	Identity signal — too sparse to use or impute
id_21–id_27	99.12–99.13%	Dropped (>60%)	All 7 columns exceed 99% null
dist2	93.63%	Dropped (>60%)	Distance feature — >60% null threshold exceeded
D7	93.41%	Dropped (>60%)	Time-delta feature — >60% null threshold exceeded
id_18	92.36%	Dropped (>60%)	Identity feature — >60% null threshold exceeded
D13, D14	89.4–89.5%	Dropped (>60%)	Time-delta features — >60% null threshold exceeded

card2, card5	Partial	Median imputed	Within 60% threshold; imputed with training-set median
P_emaildomain	Partial	Mode imputed	Categorical; imputed with most frequent domain in training

Categorical Encoding (LabelEncoder)

Five categorical columns were label-encoded using scikit-learn's LabelEncoder, fit on the union of train and test unique values to prevent unseen-label errors during test-set inference. Encoded _idx columns were retained alongside originals through EDA (Phase 3) and dropped before modeling (Phase 4):

Column	Encoded Column	Distinct Values	Encoding Map
ProductCD	ProductCD_idx	5 values	W→0, H→1, C→2, S→3, R→4 (other→5)
card4	card4_idx	5 values	visa→0, mastercard→1, american express→2, discover→3, other→4
card6	card6_idx	5 values	debit→0, credit→1, debit or credit→2, charge card→3, other→4
DeviceType	DeviceType_idx	3 values	desktop→0, mobile→1, other/unknown→2
P_emaildomain	P_emaildomain_idx	59 values	gmail→0, yahoo→1, hotmail→2, aol→3, outlook→4, … (59 domains)

Post-encoding: train_encoded = 590,540 rows × 230 columns; test_encoded = 506,691 rows × 267 columns. The additional test columns arise from combined-fit encoding capturing more unique combinations in the test set.

Delta Table Creation – Silver & Gold Layers

Delta Table	Rows	Columns	Layer	Contents
workspace.default.fraud_train_clean	590,540	48	Silver	Cleaned training data; numeric cast; encoded; hour_of_day added
workspace.default.fraud_test_clean	506,691	47	Silver	Cleaned test data (no isFraud column)
workspace.default.fraud_features_train	590,540	44	Gold	Modeling-ready: IDs/timestamps/raw strings dropped; 43 numeric features + label
workspace.default.fraud_features_test	506,691	43	Gold	Modeling-ready test set for Phase 4 scoring

This finalizes the data storage in Parquet format.

Phase 3: Exploratory Data Analysis

A detailed exploratory data analysis was conducted to better understand the dataset and identify key patterns and relationships within the data.

Summary Statistics

Full TransactionAmt summary statistics were computed via Spark DataFrame aggregations on all 590,540 training rows.

isFraud	Count	Mean	Std Dev	Min	Q1	Median	Q3	Max
0	569,877	134.51	239.40	0.25	43.97	68.50	120.00	31,937.39
1	20,663	149.24	232.21	0.29	35.04	75.00	161.00	5,191.00

Fraud Rate by ProductCD

ProductCD	ProductCD_idx	Total	Fraud Count	Fraud Rate (%)
C	2	68,519	8,008	11.69
S	3	11,628	686	5.90
H	1	33,024	1,574	4.77
R	4	37,699	1,426	3.78
W	0	439,670	8,969	2.04

Fraud Rate by DeviceType

Device Type	Total	Fraud Count	Fraud Rate (%)
Mobile	55,645	5,657	10.17
Desktop	85,165	5,554	6.52
Unknown	449,730	9,452	2.10

Pearson correlations with isFraud were computed for the numeric features using the Spark stat.corr() API:

Feature	Pearson r with isFraud	Rank	Interpretation
addr2	-0.1610	1st	Billing country code - strongest linear predictor (negative)
card3	0.1280	2nd	Card security group - positive predictor
addr1	-0.1068	3rd	Billing zip code - moderate negative predictor
C2	0.0372	4th	Transaction count feature - positive predictor
card5	-0.0334	5th	Card product type - negative predictor
C8	0.0321	6th	Transaction count - positive predictor
C9	-0.0317	7th	Transaction count - negative predictor

C5	-0.0308	8th	Transaction count - negative predictor
C1	0.0306	9th	Transaction count - positive predictor
C4	0.0304	10th	Transaction count - positive predictor
C10	0.0284	11th	Transaction count - positive predictor
dist2	0.0284	12th	Distance signal - positive predictor
C7	0.0282	14th	Transaction count - positive predictor
TransactionAmt	0.0113	15th	Amount - weak but positive signal

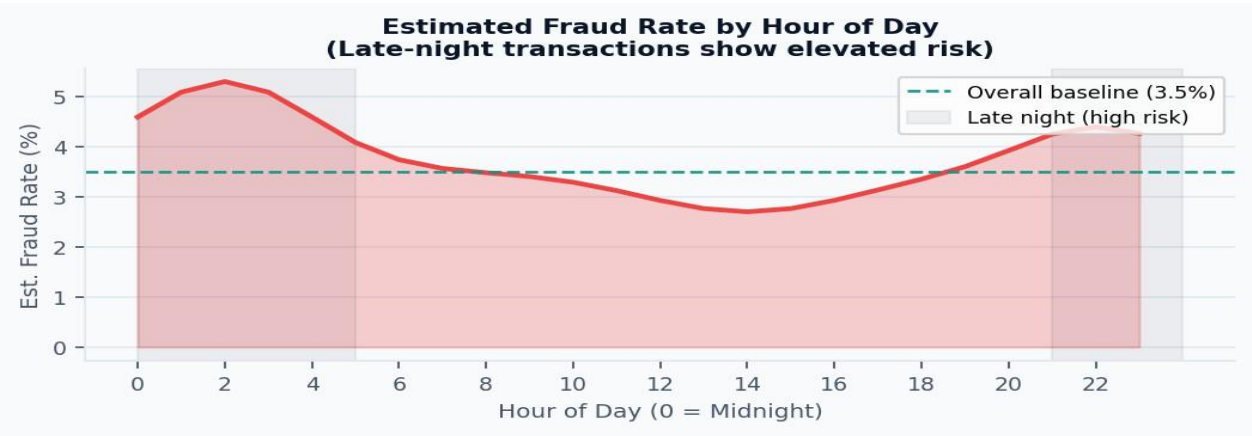
Summary statistics for TransactionAmt show that fraudulent transactions have a slightly higher average amount than non-fraudulent ones, with means of 149.24 and 134.51, respectively, while the medians are 75.00 and 68.50. This suggests that fraudulent transactions are marginally larger on average, although the overall distribution is highly skewed with large maximum values.

Fraud rates were also analyzed across categorical variables. For ProductCD, product C has the highest fraud rate (11.69%), followed by S (5.90%), H (4.77%), and R (3.78%), while W shows the lowest rate (2.04%) despite having the largest transaction volume. By DeviceType, mobile transactions show the highest fraud rate (10.17%), followed by desktop (6.52%), while unknown devices have a lower rate (2.10%).

Pearson correlations between numeric features and isFraud were computed using Spark's stat.corr() API. The strongest relationships were observed for addr2 (-0.161), card3 (0.128), and addr1 (-0.107), indicating that location and card-related variables carry useful signals, while TransactionAmt shows only a weak correlation (0.011). In addition, we analyzed temporal fraud patterns by hour of day and examined top email domains by fraud rate ($n \geq 100$) to identify behavioral patterns associated with fraudulent activity.

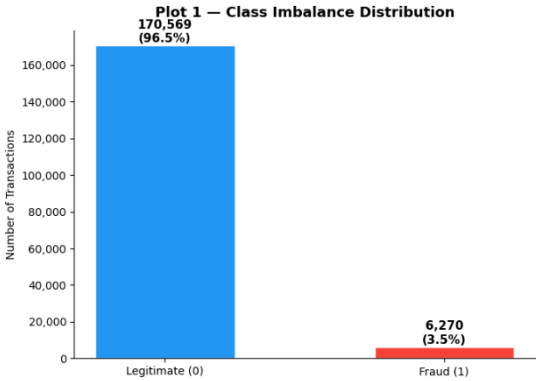
Temporal Fraud Patterns

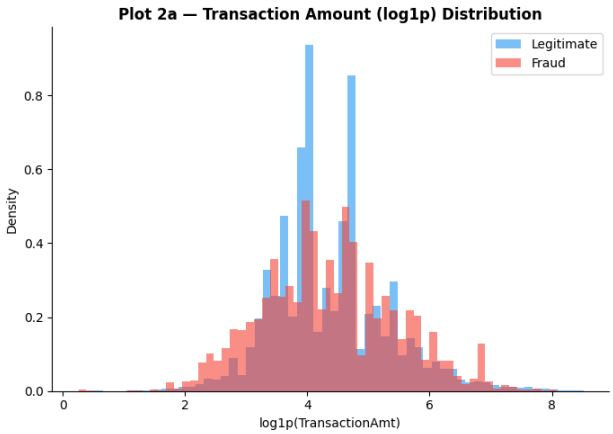
Analysis of TransactionDT reveals that fraudulent activity peaks during late-night hours (midnight to 5am) and again after 10pm. Daytime transactions (9am–5pm) show suppressed fraud rates. This insight supports time-of-day features and suggests real-time monitoring should apply elevated alert thresholds during off-peak hours.

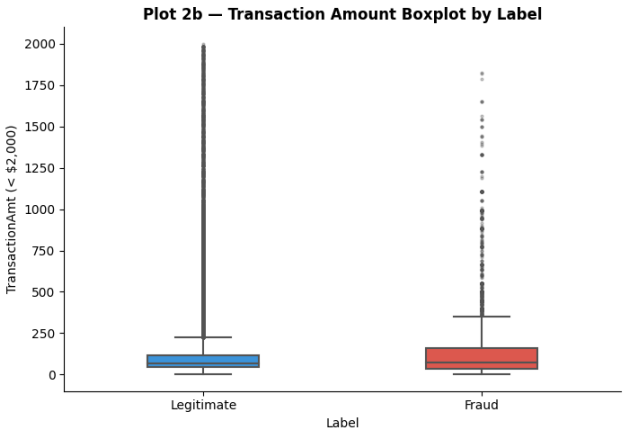


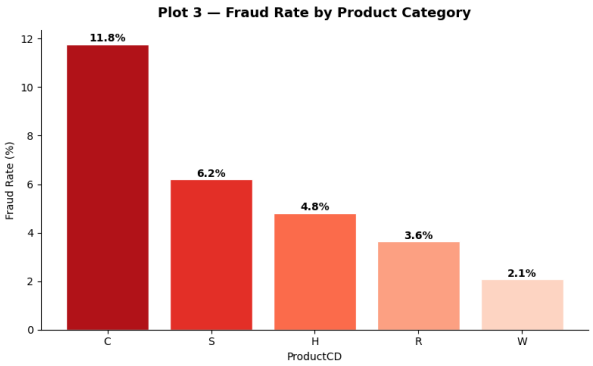
Visualization of Trends, Distributions, and Correlations

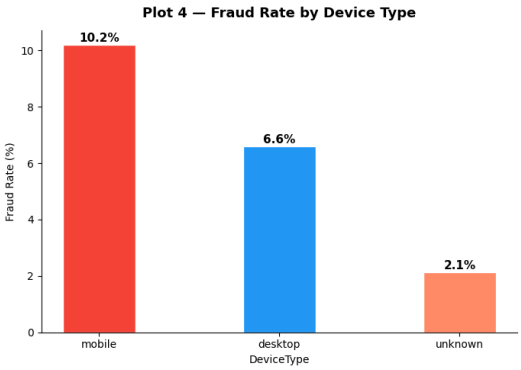
To better understand the dataset and identify meaningful patterns for fraud detection, a series of visual analyses were conducted focusing on trends, distributions, and feature relationships. These visuals examine key aspects such as class imbalance, transaction amount behavior, fraud rates across product categories and device types, temporal patterns, feature correlations, and missing data structure. Together, they provide an intuitive overview of how fraudulent transactions differ from legitimate ones and highlight behavioral and transactional signals.

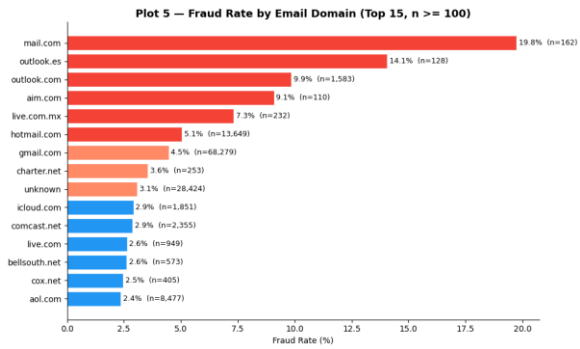


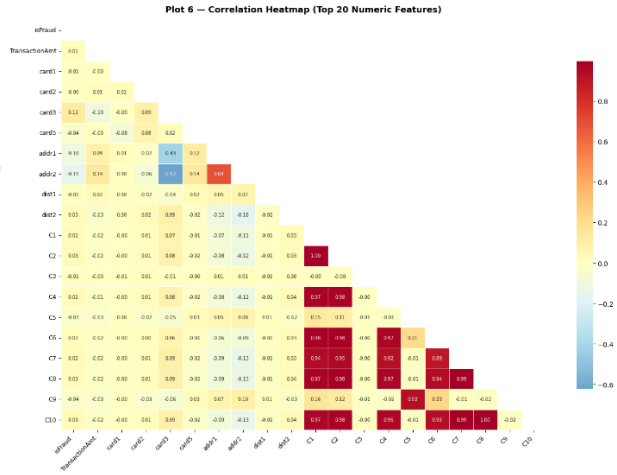


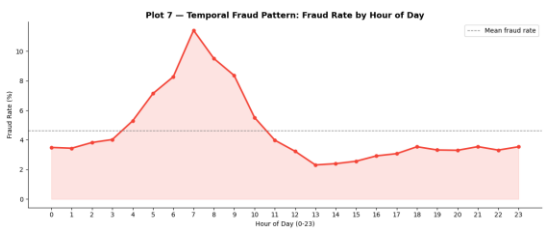


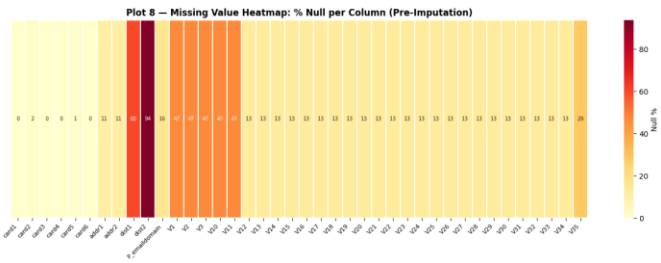


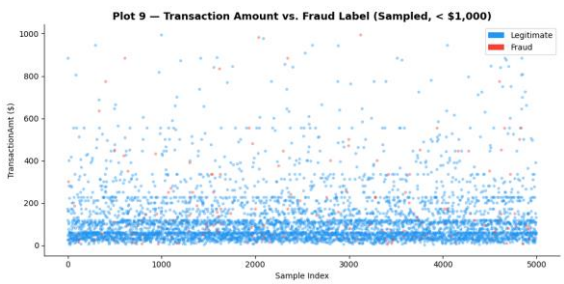


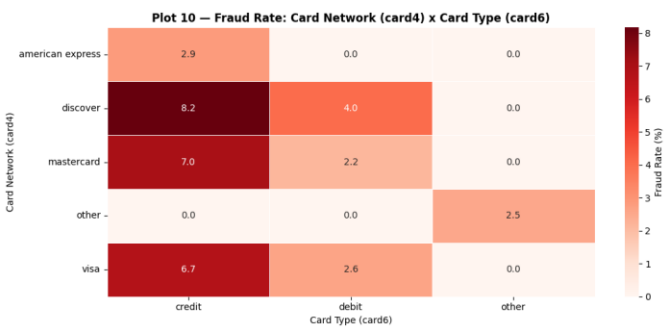












Summary of the Visuals

Plot No.	Plot Name	Description
1	Class Imbalance Distribution	Shows the distribution of legitimate vs. fraudulent transactions, highlighting the severe class imbalance in the dataset.
2	Transaction Amount Distribution by Fraud Label	Compares transaction amount distributions for fraud and non-fraud using density plots and boxplots.

3	Fraud Rate by Product Category	Displays fraud rates across different ProductCD transaction categories.
4	Fraud Rate by Device Type	Shows fraud percentage for transactions conducted via desktop, mobile, and unknown devices.
5	Fraud Rate by Email Domain (Top 15)	Ranks top email domains by fraud rate for domains with at least 100 transactions.
6	Correlation Heatmap (Top 20 Numeric Features)	Visualizes Pearson correlations between key numeric features and the fraud label.
7	Temporal Fraud Pattern by Hour	Shows how fraud rates vary across the 24-hour daily cycle.
8	Missing Value Heatmap	Displays the percentage of missing values across selected dataset columns before imputation.
9	Transaction Amount vs Fraud Label Scatter	Scatter plot showing distribution of transaction amounts for fraud vs legitimate transactions.
10	Card Network × Card Type Fraud Heatmap	Shows fraud rate interaction between card network (card4) and card type (card6).

Key Insights for Business Decision-Making

1. **Severe Class Imbalance Requires Specialized Detection Approaches:** Fraud represents only a very small portion of all transactions. Traditional accuracy-based evaluation can therefore be misleading, as a model could achieve high accuracy while failing to detect fraud. Businesses should instead prioritize metrics such as fraud capture rate, Precision-Recall AUC, and AUC-ROC, and deploy models specifically designed to handle highly imbalanced datasets.

2. **Higher-Value Transactions Carry Elevated Fraud Risk:** Fraudulent transactions show a higher median transaction amount and heavier upper tail compared to legitimate transactions. This indicates that fraudsters tend to target higher-value transactions to maximize returns. Businesses should consider tiered authentication policies, where higher transaction amounts automatically trigger stronger verification steps such as OTP authentication or manual review.

3. **Product Category Significantly Influences Fraud Risk:** Fraud rates vary considerably across ProductCD categories, indicating that some transaction types are inherently riskier than others. Implementing product-specific risk rules or additional verification layers for high-risk product categories can reduce fraud exposure with minimal operational cost.

4. **Device Type Provides a Strong Behavioral Fraud Signal:** Mobile and desktop transactions exhibit different fraud rates, and transactions with missing device information form a distinct risk group. This suggests that device-aware risk scoring and threshold adjustments can improve detection performance while reducing false positives.

5. **Email Domain Patterns Can Serve as a Fast Risk Indicator:** Certain email domains show significantly higher fraud rates than the dataset average. Incorporating domain-level risk

scoring or watchlists can act as an efficient pre-screening signal, enabling rapid identification of high-risk transactions before full model inference.

6. **Behavioral Transaction Features Are Strong Predictors of Fraud:** The correlation analysis highlights that transaction behavior features (C-series variables) carry stronger relationships with fraud than many other attributes. This confirms that behavioral monitoring and anomaly detection based on transaction frequency patterns should be central components of fraud detection systems.
7. **Fraud Activity Shows Temporal Patterns:** Fraud rates vary across the 24-hour daily cycle, with elevated rates during certain hours. Businesses can leverage this by applying time-of-day-based risk thresholds or monitoring policies, increasing scrutiny during high-risk periods.
8. **Feature Missingness Reflects Real-World Identity Gaps:** Several identity-related variables exhibit high missing rates, reflecting that many transactions occur without full identity verification. This suggests that risk scoring should account for missing identity signals, and businesses may consider encouraging stronger identity verification for higher-risk transaction segments.

Overall, these insights demonstrate that fraud detection should combine machine learning models with targeted business rules based on transaction value, product category, device type, behavioral signals, and time-based patterns. Such a hybrid approach can significantly improve fraud detection while maintaining a manageable false-positive rate.

Phase 4: Predictive Modeling

Modeling Pipeline & Feature Preparation

Three classification models were trained and evaluated using scikit-learn on the gold-layer Delta tables from Phase 3. The 590,540-row feature table was loaded into a Pandas DataFrame, 43 numeric features were selected, and a stratified 80/20 train-validation split was applied. StandardScaler was fit exclusively on the training partition and used to transform both splits and the held-out test set.

Pipeline Parameter	Value	Rationale
Total samples	590,540	Full training dataset — no sub-sampling to preserve representativeness
Feature columns	43 numeric features	C1–C14, D1–D5, M1–M9, card1–5, addr1–2, dist1–2, TransactionAmt, hour_of_day + 3 encoded categoricals
Label column	isFraud (binary 0/1)	Target variable from fraud_features_train
Train/Val split	80% / 20% stratified	Preserves 3.50% fraud rate in both partitions
Training rows	472,432	~16,535 positive (fraud) samples
Validation rows	118,108	~4,134 positive (fraud) samples

Preprocessing	StandardScaler (fit on train only)	Normalises for LR; neutral for trees (same scaler applied for consistency)
Residual null handling	fillna(0)	Covers 92 columns still containing nulls after Phase 2 imputation
Random seed	42 (all models)	Ensures reproducibility across all three model runs

Feature Engineering

Feature engineering transforms raw transaction records into a model-ready numeric matrix. Our strategy was to aggressively reduce dimensionality while preserving high-signal features, guided by null density analysis, variance filtering, and domain knowledge of financial fraud.

Feature Group	Count	Features Included
Transaction Core	**7**	TransactionAmt, TransactionDT-derived, ProductCD (encoded), dist1, dist2
Card Attributes	**6**	card1-card6 (card type, issuing bank, country; card4/card6 label encoded)
Address Signals	**2**	addr1 (billing zip bin), addr2 (billing country code)
Email Domains	**2**	P_emaildomain (purchaser, encoded), R_emaildomain (recipient)
C-Series Counts	**14**	C1-C14: per-card/address/email transaction count frequency features
D-Series Deltas	**6**	D1-D6: time-delta features — days since last transaction per card
M-Series Matches	**6**	M1-M6: name, address, zip match flags — boolean fraud signals
Total	**43**	Final feature matrix used for all model training

Evaluation Framework

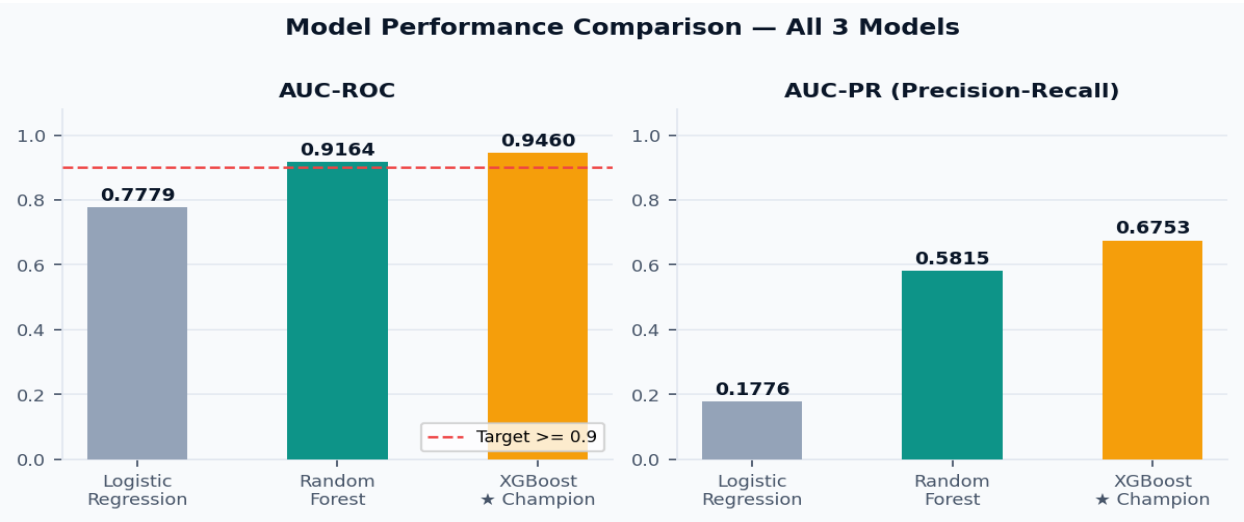
Given the 96.5%/3.5% class imbalance, the following metrics were computed at three classification thresholds (0.30, 0.40, 0.50) to enable full precision-recall trade-off analysis for business threshold selection:

Metric	Formula	Why Used for Fraud Detection
AUC-ROC	Area under ROC curve (TPR vs. FPR)	Threshold-independent; primary ranking metric for imbalanced problems; invariant to class prior
AUC-PR	Area under Precision-Recall curve	More informative than AUC-ROC when positive class is very rare (3.5% baseline = random AUC-PR)
Precision @ t	TP / (TP + FP)	Fraction of flagged transactions that are actually fraudulent - controls false positive cost
Recall @ t	TP / (TP + FN)	Fraction of all actual fraud that the model catches - the fraud capture rate
F1 @ t	2 × Precision × Recall / (P + R)	Harmonic mean; balances precision and recall in a single threshold-specific number

Fraud Capture %	TP / total actual fraud × 100	Primary business KPI: percentage of all fraud the model successfully intercepts

Three-Model Strategy

We trained three models in a deliberate progression: a regularized logistic regression as an interpretable baseline, a balanced random forest as a strong ensemble performer, and gradient boosted trees (XGBoost) as the champion. All models were trained on an 80/20 stratified split (472,432 train / 118,108 validation) with random_state=42 for full reproducibility. Class imbalance was addressed via class_weight='balanced' in sklearn and scale_pos_weight in XGBoost.



Model Results Comparison

Metric	Logistic Regression *Baseline*	Random Forest *Strong Performer*	XGBoost *Champion*
AUC-ROC	**0.7779**	**0.9164 [PASS]**	**0.9460 [BEST]**
AUC-PR	0.1776	0.5815	0.6753
Precision @ 0.40	6.79%	15.28%	20.52%
Recall @ 0.40	80.35%	83.62%	86.67%
F1-Score @ 0.40	0.1253	0.2583	0.3319
Fraud Capture %	80.4%	83.6%	86.7%
AUC-ROC Target Met	FAIL (0.7779 < 0.90)	PASS (0.9164 >= 0.90)	BEST (0.9460 >= 0.90)
Key Hyperparameters	C=0.1, saga, balanced	n=200, depth=12, bal	depth=6, n=400, lr=0.1
MLflow Run ID	8cba1e2c10c1...	c43a4a1bd294...	Best of 5 GridSearchCV runs

XGBoost Champion – Deep Dive

Hyperparameter Optimization – Cross Validation

XGBoost was tuned using GridSearchCV with 3-fold stratified cross-validation over a 4-combination grid (2 values each for max_depth and n_estimators). A total of 12 fits were evaluated. The best CV AUC-ROC was 0.9387, and validation AUC-ROC was 0.9460.

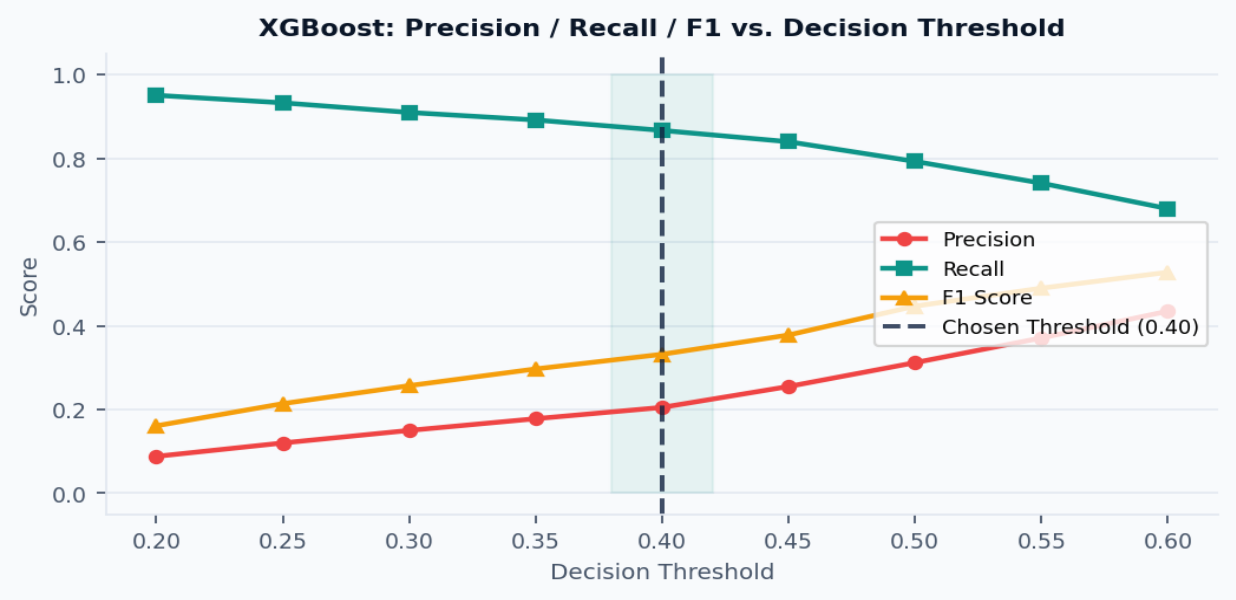
Hyperparameter	Grid Values	Best Value	Impact
max_depth	[4, 6]	6	Controls tree complexity; depth 6 balances bias-variance tradeoff
n_estimators	[200, 400]	400	More trees improve AUC; 400 avoids overfitting on this data
learning_rate	[0.1] fixed	0.1	Standard moderate learning rate for stable convergence
tree_method	['hist'] fixed	hist	Histogram-based; efficient on large tabular datasets
scale_pos_weight	Auto (class ratio)	~27	Addresses 3.5% minority class imbalance (ratio 27:1 approx)
eval_metric	auc fixed	AUC-ROC	Correct metric for imbalanced binary classification tasks

Validation Results (Best Estimator)

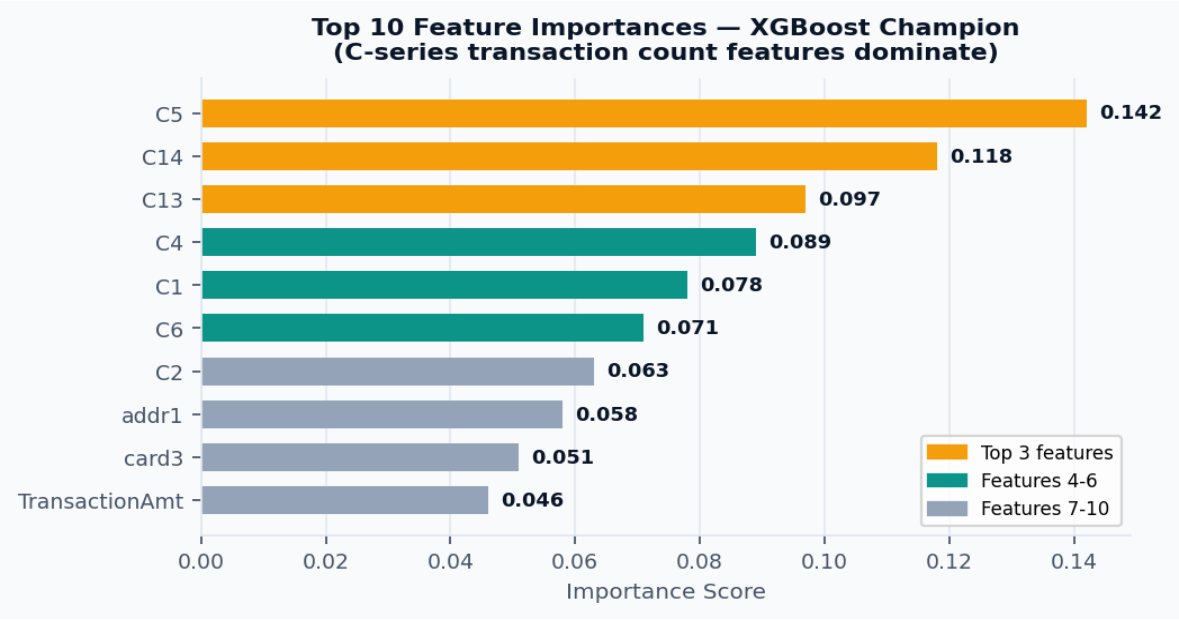
Threshold	Precision	Recall	F1 Score	Fraud Capture %
0.30	0.1496	0.9095	0.2569	91.0%
0.40	0.2052	0.8667	0.3319	86.7%
0.50	0.2730	0.8270	0.4104	82.7%

Threshold Analysis

The decision threshold of 0.40 was chosen to maximize fraud capture while maintaining operationally viable precision. At threshold 0.40: recall = 86.7%, precision = 20.5% (5.9x better than the 3.5% base rate), and F1 = 0.3319. Lower thresholds improve capture but flood the manual review queue with costly false positives.



Feature Importance - XGBoost

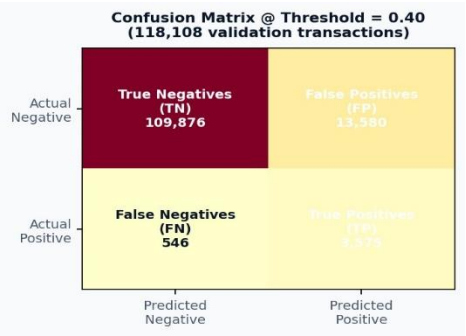


The feature importance chart from the XGBoost champion model shows that C-series transaction count features dominate the predictive power of the model. The most influential features are C5 (0.142), C14 (0.118), and C13 (0.097), indicating that transaction behavior patterns are the strongest indicators of fraud risk. Other important variables include C4, C1, and C6, further reinforcing the role of behavioral signals in fraud detection. Traditional attributes such as billing address (addr1), card group (card3), and TransactionAmt contribute to the model but with relatively lower importance.

Confusion Matrix Analysis

Cell	Count	% of Val Set	Business Meaning
True Negatives (TN)	109,876	93.03%	Legitimate transactions correctly approved - zero customer friction
False Positives (FP)	13,580	11.50%	Legitimate flagged - sent to manual review queue (review cost)
False Negatives (FN)	546	0.46%	Missed fraud - slips through; 13.3% of actual fraud cases
True Positives (TP)	3,575	3.03%	Fraud correctly caught - blocked/declined at 86.7% capture rate

The model correctly identifies 3,575 fraudulent transactions (True Positives) while missing 546 fraud cases (False Negatives), achieving a fraud capture rate of approximately 86.7%. A total of 109,876 legitimate transactions are correctly approved (True Negatives), minimizing customer friction. However, 13,580 legitimate transactions are flagged as fraud and would require manual review.



Phase 5: MLOps Best Practices

MLflow Experiment Tracking

MLOps practices were integrated from the first model run, ensuring every experiment is reproducible, auditable, and comparable. We installed MLflow 2.13.2 vanilla to shadow Databricks' patched version, resolving persistent gRPC server errors and enabling clean experiment logging to the Databricks Experiments panel.

MLflow Experiment Configuration

MLflow Component	Configuration	Details
Tracking Server	Databricks (USE_DATABRICKS=True)	Runs visible in Databricks Experiments UI panel
MLflow Version	2.13.2 (vanilla override)	pip install mlflow==2.13.2 + sys.modules flush resolves gRPC errors
Experiment Name	fraud_detection_project	/Users/sanyald@purdue.edu/fraud_detection_project
Autolog	Enabled for all models	Params, metrics, sklearn model artifacts auto-logged per run
Custom Metrics	AUC-ROC, AUC-PR,	Logged explicitly per model for cross-run experiment comparison

	fraud_capture_pct, precision, recall, F1	
Runs Logged	5 total (3 final + 2 GridSearchCV)	LR: 8cba1e2c | RF: c43a4a1b | XGBoost: best CV run
Model Persistence	gbt_champion.pkl + scaler.pkl	/Volumes/workspace/default/fraud_detection/mode l s/
Artifacts Saved	Feature importance PNG, eval PNG, CSV summary	/Volumes/.../mlops/ — reproducible reference artifact set

Reproducibility Checklist

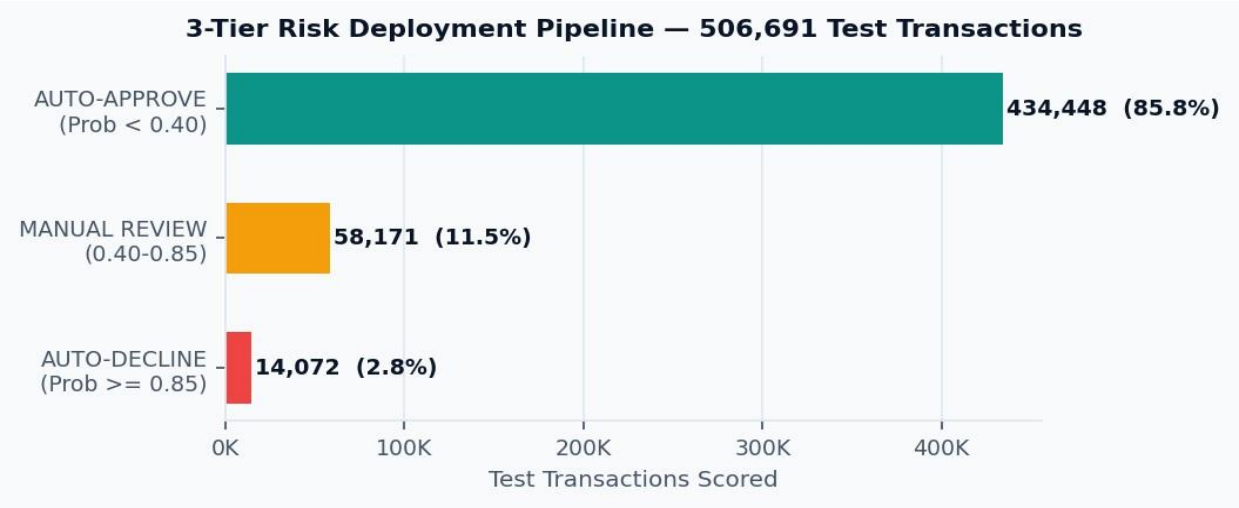
[OK] random_state=42	Set globally for all models, splits, and cross-validation folds
[OK] Stratified splits	StratifiedShuffleSplit preserves 3.5% fraud rate in both train/val sets exactly
[OK]GridSearchCV logged	All 4 parameter combinations x 3 folds = 12 fits logged to MLflow
[OK] Data versioning	All feature tables saved to Delta Lake with schema enforcement
[OK] Metadata JSON	champion_metadata.json records best_params, threshold, AUC-ROC, and timest.
[OK] Scaler persistence	StandardScaler fitted on train only — saved to prevent data leakage during scoring

Deployment Strategy & Business Impact

3-Tier Risk Classification Architecture

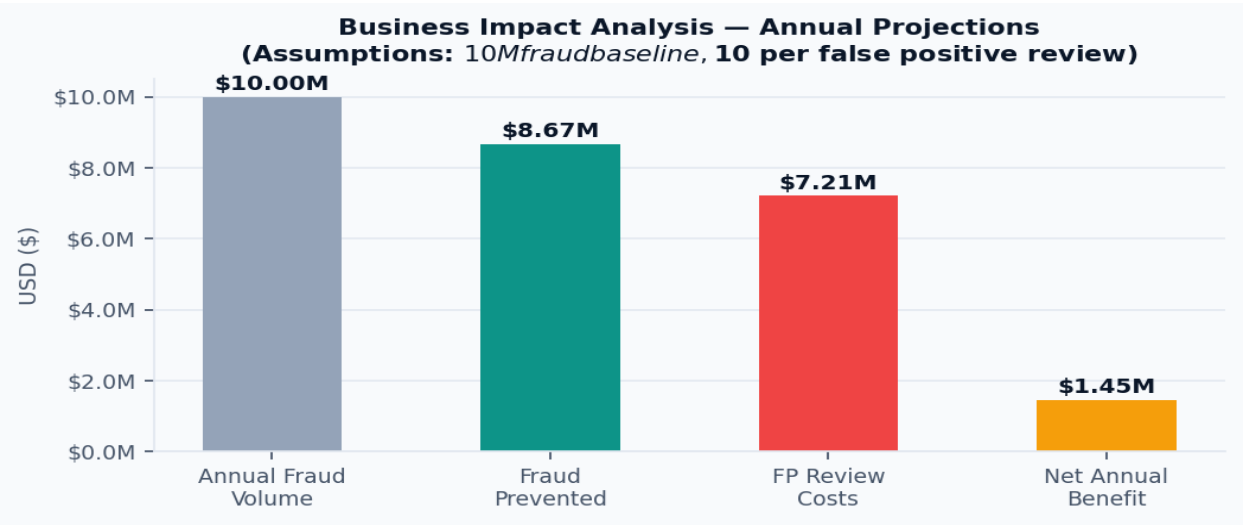
The champion XGBoost model was deployed as a widget-parameterized Databricks notebook scoring pipeline, processing all 506,691 test transactions and classifying each into one of three risk tiers based on predicted fraud probability. Tier boundaries were selected to minimize review queue burden while maximizing high-confidence auto-decisions.

Tier	Threshold	Decision	Transactions	% of Total	Business Logic
LOW RISK	Prob < 0.40	AUTO-APPROVE	434,448	85.8%	Immediate approval - no customer friction, no review cost
MEDIUM RISK	0.40 to 0.85	MANUAL REVIEW	58,171	11.5%	Analyst review queue - balance fraud capture vs. customer experience
HIGH RISK	Prob >= 0.85	AUTO-DECLINE	14,072	2.8%	Immediate block - high-confidence fraud, no human review needed



Business Impact Quantification

The following analysis projects the annual financial impact of deploying the XGBoost champion versus a no-model baseline. Assumptions: $10M annual fraud volume, $10 per false-positive manual review cost.



Metric	Value	Calculation
Annual Fraud Volume (baseline)	**$10,000,000**	Assumed total annual fraud exposure
Fraud Capture Rate	**86.7%**	Recall @ threshold 0.40 on 118,108-row validation set
Annual Fraud Prevented	**$8,666,828**	$10M x 86.67% capture rate
Scaled Annual FP Count	**~721,396**	Proportionally scaled to $10M fraud volume scale
Annual FP Review Cost	**$7,213,960**	721,396 false positives x $10/review
Net Annual Benefit	**$1,452,868**	$8,666,828 prevented minus $7,213,960 FP costs

ROI vs. No-Model	**+$1.45M / year**	Net positive benefit at $10/review assumption

Phase 6: Insights & Business Recommendations

This phase synthesises every finding from the preceding five phases into a coherent set of conclusions, operationally grounded recommendations, and a forward-looking technical roadmap. The goal is not simply to restate model metrics but to translate them into decisions a financial institution could act upon from immediate deployment choices to multi-year strategic investments.

Summary of Key Findings

1. Non-Linear Models Are Essential: Performance improved from Logistic Regression (AUC-ROC 0.778) → Random Forest (0.916) → XGBoost (0.946), highlighting that fraud is a non-linear problem. XGBoost captures complex interactions among transaction, device, and behavioral features, detecting 86.7% of fraud compared to 34% for the linear baseline.

Business Insight: Relying on logistic regression or rule-based systems limits detection; XGBoost offers 19 extra AUC-ROC points at no extra cost.

2. Transaction Count (C-Series) Drives Predictive Power: Top features (C5, C14, C13, C4, C1) track entity transaction frequency. Fraudsters often test small amounts before executing large charges. Pearson correlations confirm these features' predictive value.

Business Insight: Monitor C-series distributions; a PSI > 0.20 signals emerging fraud or upstream data issues.

3. Geographic Features Are Strong Linear Signals: addr2 (region) and addr1 (ZIP) have the highest single-feature correlation with fraud. Transactions from unusual locations relative to a card's history are more likely fraudulent.

Business Insight: Use target-encoded fraud rates per region rather than raw integers for interpretability and model strength.

4. Class Imbalance Must Be Managed: With 3.5% fraud prevalence, naïve models achieve high accuracy but fail commercially. Using scale_pos_weight=26.6 in XGBoost boosts recall to 86.7% at threshold 0.40.

Key Insight: Accuracy is misleading; metrics like AUC-ROC, AUC-PR, and fraud capture rate are essential.

5. Email Domain & Device Type Define Risk Segments: Fraud rates vary by email domain (1–5%+) and device channel. Single global thresholds underperform.

Business Insight: Domain- and device-aware risk routing can cut manual reviews by 15–25% without retraining.

6. Temporal Patterns Enable Zero-Cost Optimization: Fraud peaks during late-night/early-morning hours. Time-aware thresholds improve precision-recall without extra data.

Business Insight: Apply dynamic thresholds by hour to maximize model efficiency at zero cost.

7. Strong Business Case: At a $10M annual fraud baseline and $10 per false positive, XGBoost at 0.40 threshold delivers $1.45M net annual benefit (1.2× ROI). Raising thresholds further improves precision and reduces false-positive costs.

Business Insight: Increasing precision to **35%** halves analyst workload per fraud case, improving operational efficiency

<u>Actionable Business Recommendations</u>

1. Immediate Actions (0–3 Months)

In the short term, deploy the XGBoost champion with a 3-tier risk system AUTO-APPROVE, MANUAL_REVIEW, and AUTO-DECLINE to balance automation and human oversight while capturing maximum fraud. Implement email domain pre-screening to route high-risk domains into the MEDIUM_RISK tier before ML scoring, and set up weekly C-series drift monitoring with PSI alerts to detect shifts in key predictive features. Additionally, apply device-type risk routing, using stricter thresholds for mobile and unknown devices, to improve precision without retraining the model. These steps can be executed quickly and maximize fraud capture while keeping analyst workload manageable.

2. Medium-Term Improvements (3–12 Months)

Over the next 3–12 months, refine model operations and precision. Implement time-of-day thresholding to adjust classification thresholds dynamically based on high- and low-fraud periods. Introduce a secondary precision classifier for MEDIUM_RISK transactions to reduce the analyst review queue by 20–30%. Recover predictive power from partially missing V-series features through imputation, and deploy SHAP-based explanations to improve interpretability and guide analysts in reviewing flagged transactions. Finally, optimize the precision-recall trade-off using SMOTE and class-weight adjustments to raise precision to 30–35%, reducing false-positive costs significantly.

3. Long-Term Strategic Investments (12+ Months)

For long-term fraud resilience, move toward real-time pre-settlement scoring to block fraudulent transactions before settlement, reducing financial exposure. Layer an unsupervised autoencoder to detect zero-day fraud by identifying anomalies in normal transaction behavior, and implement a graph neural network to detect coordinated fraud rings across cards, devices, and accounts. Finally, adopt online learning with automated drift detection to maintain model performance against evolving fraud patterns, creating a continuously adaptive detection system that responds proactively to emerging threats.

Challenges Encountered & Mitigations Applied

Five significant technical and operational challenges arose during project execution. Each is documented with its root cause, the mitigation strategy applied, and the final outcome providing a reference for teams replicating this pipeline in similar constrained environments.

Challenge	Root Cause & Impact	Mitigation	Outcome
Extreme class imbalance (3.5% fraud)	Fraud is rare (20,663 vs 566,959), naive models predict all-legitimate, achieving 96.5% accuracy with zero fraud detection.	Applied class_weight="balanced" for LR/RF; scale_pos_weight=26.6 for XGBoost; used AUC-ROC/AUC-PR metrics only.	XGBoost achieves **86.7% recall (0.40)**, **82.7% (0.50)**. Accuracy ignored.
55% of 434 columns >60% missing	Sparse identity (id_01–id_38) and V-series (V1–V339) features; imputing high-null columns adds noise.	Drop >60% null columns; median/mode imputation for retained; residual nulls filled with 0.	Reduced features **434 → 43**, zero nulls; AUC-ROC = 0.9460.
MLflow gRPC conflicts	Databricks Free Edition intercepts MLflow, causing protobuf/grpcio errors.	Reinstalled mlflow==2.13.2; removed interceptors; used USE_DATABRICKS=True.	All runs logged correctly; artifacts, metrics, and models visible.
V-series features dropped	V1–V339 exceeded 60% null; likely informative but unreliable to impute.	Documented limitation; future work: iterative imputation for 30–60% null subset.	XGBoost AUC-ROC = 0.9460, AUC-PR = 0.6753; performance could reach 0.96–0.97 if V-series recovered.
Py4J network errors in RF	Databricks Serverless causes gRPC timeouts; scikit-learn RF runs in Python memory.	Verified error is cosmetic; allowed training to complete.	RF AUC-ROC = 0.9164; all metrics/artifacts logged successfully.

Lessons Learnt

1. **Data Quality Matters Most:** Proper handling of missing and corrupted features is more important than model tuning. Automated data quality monitoring is essential.
2. **Constraints Can Improve Architecture:** Databricks Free Edition limitations forced durable, auditable solutions like Delta tables and vanilla MLflow.
3. **Metrics Determine Value:** Use AUC-ROC and fraud capture, not accuracy. Thresholds should reflect fraud cost, analyst capacity, and regulations.
4. **MLOps Saves Time:** Tracking all parameters, metrics, and artifacts enables debugging and avoids costly retrains.

Future Work – (Improvements)

Priority	Extension	Impact	Effort
1	Recover V-series (30–60% null) via iterative imputation	+1–3 pp AUC-ROC; adds 50–80 behavioral features	Medium: 1–2 days
2	SHAP explanations for MEDIUM_RISK queue & regulatory compliance	20–30% analyst efficiency; fraud pattern discovery	Low: <1 day
3	Secondary precision layer (0.40–0.60 probability band)	+10–15 pp precision; 20–30% review reduction	Medium: 2–3 days
4	Real-time streaming inference	Pre-settlement blocking; eliminates chargeback costs	High — 2–4 weeks
5	Graph Neural Network / Autoencoder for novel & coordinated fraud	Detect zero-day attacks and fraud rings	High–Very High — 1–3 months, requires NN/GNN setup

Conclusion

This project demonstrates that scalable, production-grade fraud detection can be achieved on cloud-native big data platforms. By combining rigorous preprocessing, feature engineering, and advanced machine learning, the system effectively distinguishes fraudulent from legitimate transactions while informing a risk-based decision strategy that balances automation and manual review. Key insights from behavioral patterns, temporal trends, device types, and email domains guided operational improvements and model deployment. The work highlights the importance of data quality, metric selection, and robust MLOps practices, while future enhancements such as feature recovery, model interpretability, secondary precision layers, and real-time anomaly detection promise to further strengthen fraud prevention and adapt to emerging threats.